

Chapter 9

Heap

Objectives

- Define and implement heap structures
- Understand the operation and use of the heap ADT

9-1 Basic Concepts

- A heap is a binary tree whose left and right subtrees have values less than their parents. The root of a heap is guaranteed to hold the largest node in the tree.
- Both the left and the right branches of the tree have the same properties.
- Heaps are often implemented in an array rather than a linked list. When arrays are used, we are able to calculate the location of the left and the right subtrees. Conversely, we can calculate the address of it's parent.
- The tree is complete or nearly complete. The key value of each node is greater than or equal to the key value in each of its descendents. This structure is also called max- heap.

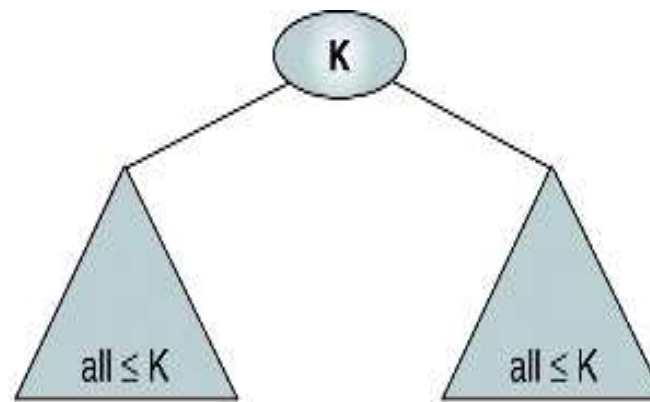
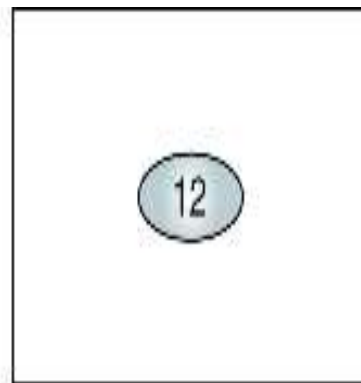
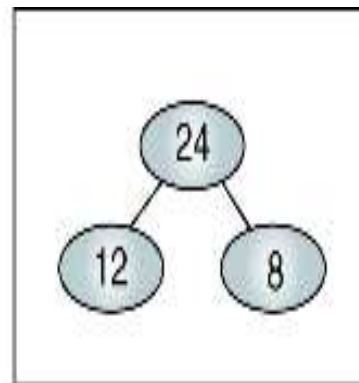


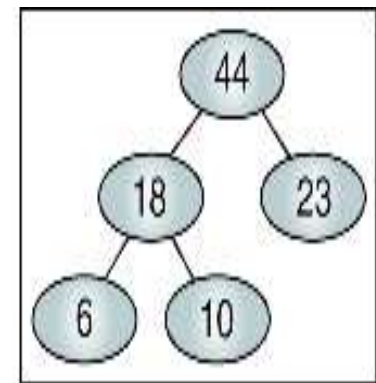
FIGURE 9-1 Heap



(a) Root-only heap

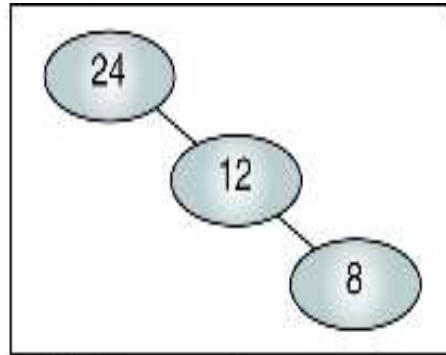


(b) Two-level heap

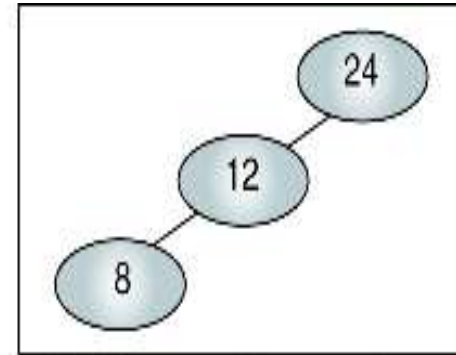


(c) Three-level heap

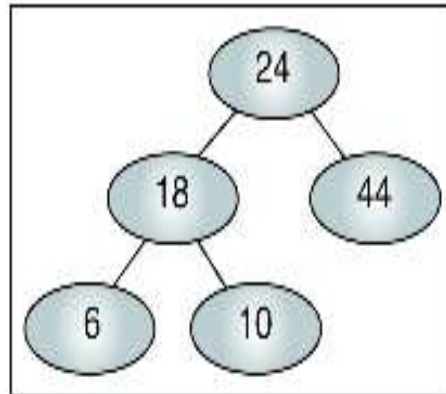
FIGURE 9-2 Heap Trees



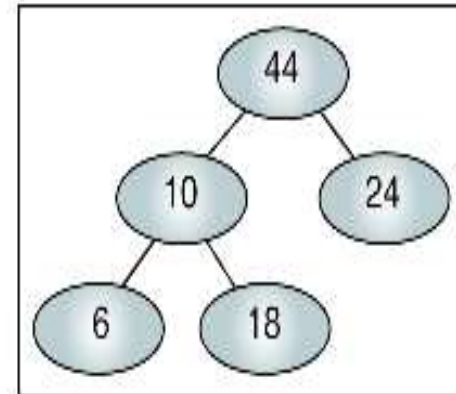
**(a) Not nearly complete
(rule 1)**



**(b) Not nearly complete
(rule 1)**



**(c) Root not largest
(rule 2)**



**(d) Subtree 10 not a heap
(rule 2)**

FIGURE 9-3 Invalid Heaps

Maintenance Operations

Two basic maintenance operations are performed on a heap.

- **Insert a heap**
- **Delete a heap**
- **Two basic algorithms are – Reheap Up and Reheap Down**

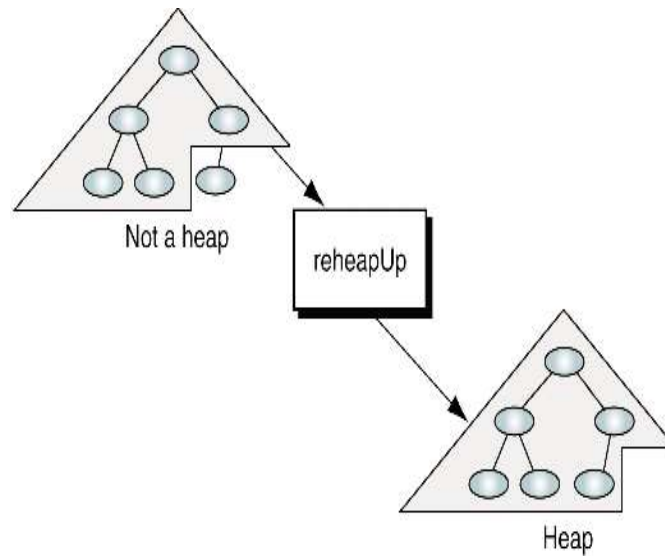
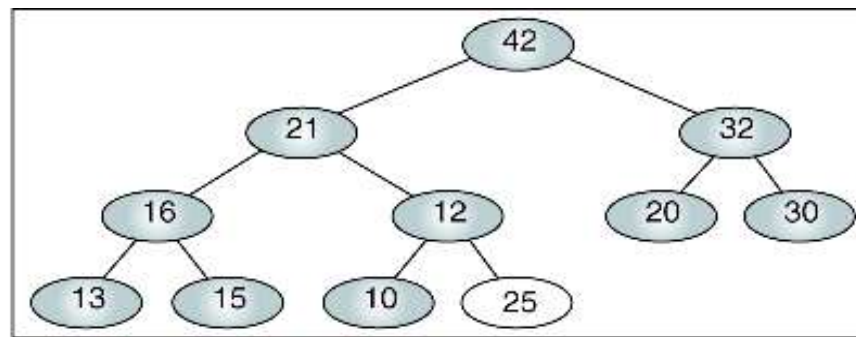
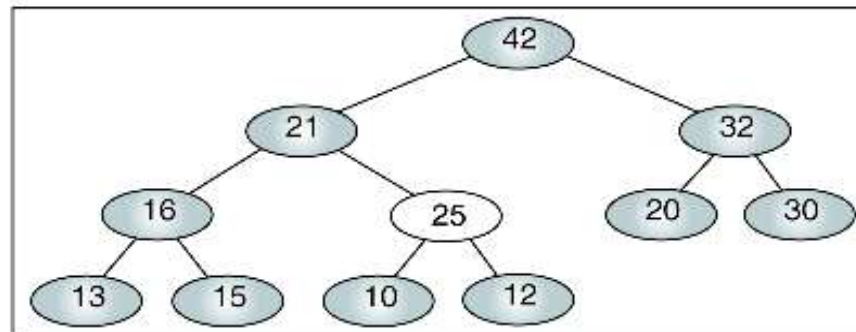


FIGURE 9-4 Reheap Up Operation

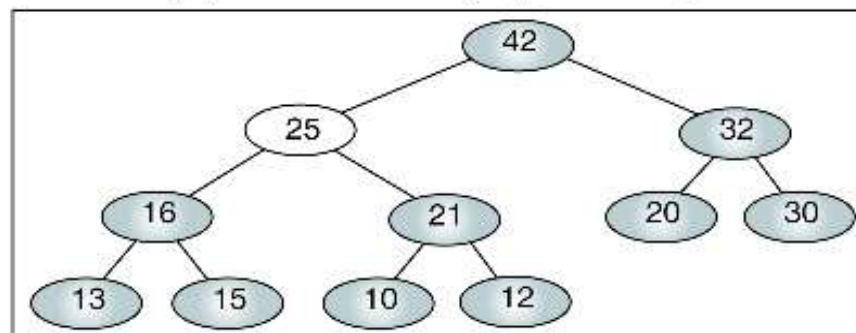
- The reheap up operation repairs the structure so that it is a heap by floating the last element up the tree until that element is in its correct location.
- Insertion takes place at a leaf at the first empty position. This may create a situation where the new node's key is larger than that of its parent. If it is, the node is floated up the tree by exchanging the child and parent keys and data.



(a) Original tree: not a heap



(b) Last element (25) moved up



(c) Moved up again: tree is a heap

FIGURE 9-5 Reheap Up Example

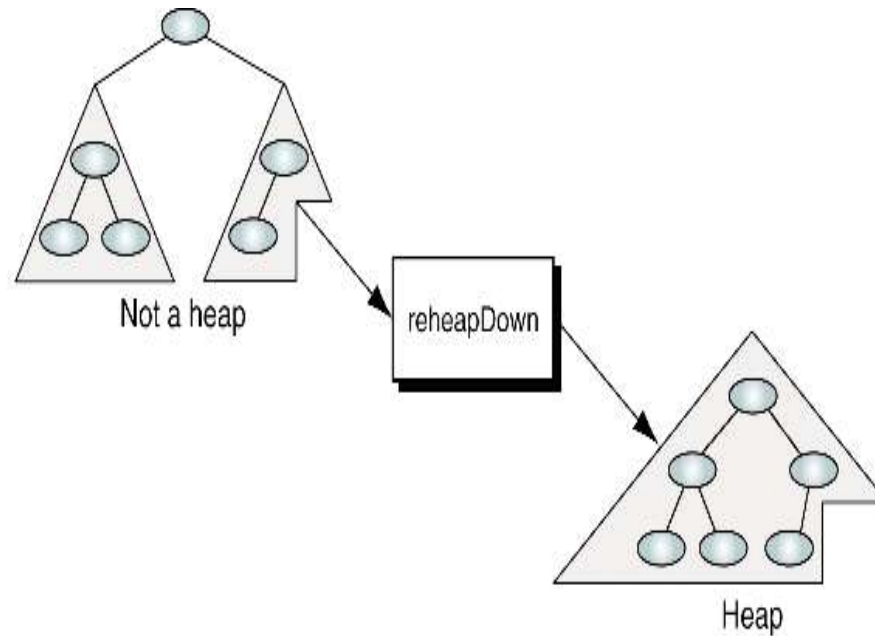
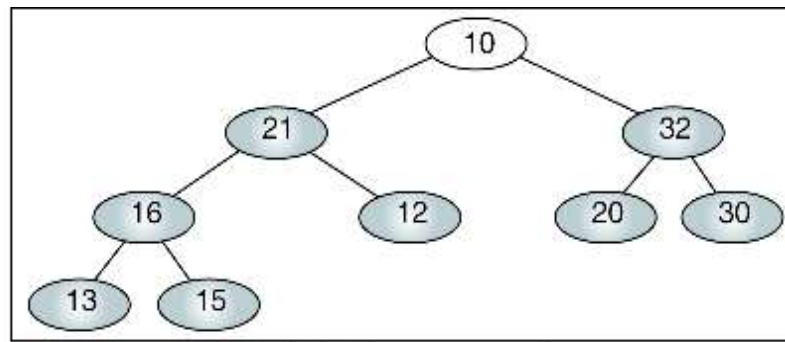
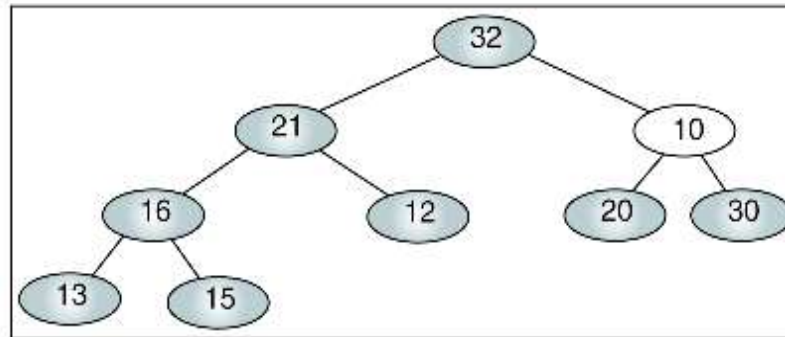


FIGURE 9-6 Reheap Down Operation

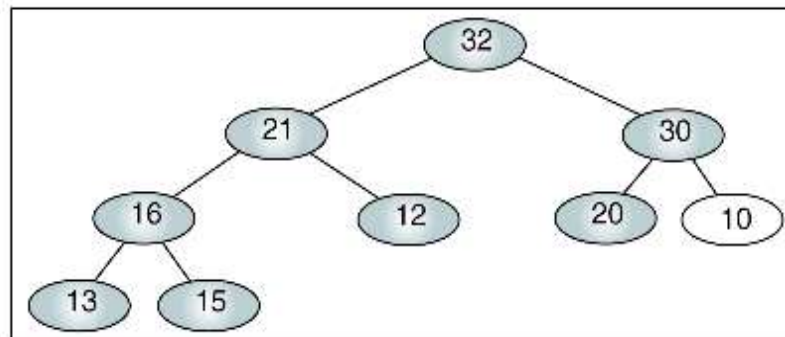
When we have a nearly complete binary tree that satisfies heap order property except in the root position. Reheap down operation reorders a broken heap by pushing the root down the tree until it is in correct position at the heap.



(a) Original tree: not a heap



(b) Root moved down (right)



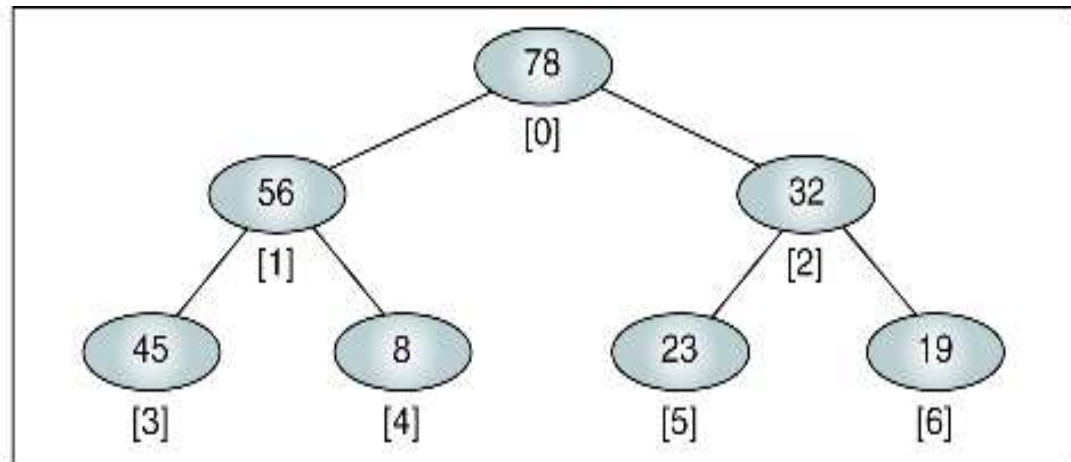
(c) Moved down again: tree is a heap

FIGURE 9-7 Reheap Down Example

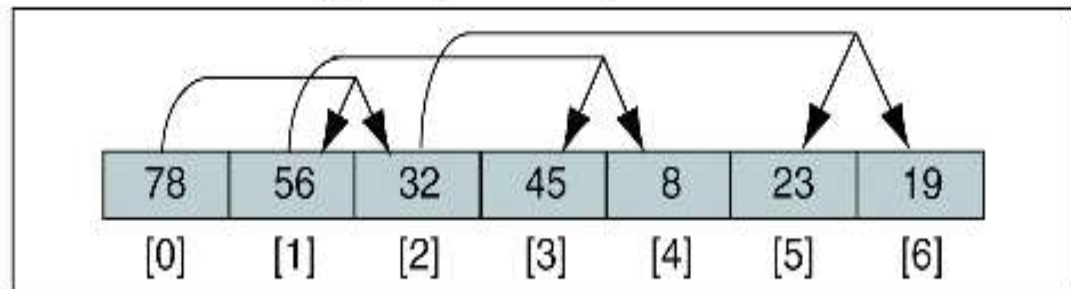
9-2 Heap Implementation

Heaps are usually implemented in an array structure. In this section we discuss and develop five heap algorithms.

- Reheap Up
- Reheap Down
- Build a Heap
- Insert a Node into a Heap
- Delete a Node from a Heap



(a) Heap in its logical form



(b) Heap in an array

FIGURE 9-8 Heaps in Arrays

ALGORITHM 9-1 Reheap Up

Algorithm reheapUp (heap, newNode)

Reestablishes heap by moving data in child up to its correct location in the heap array.

Pre heap is array containing an invalid heap

¬newNode is index location to new data in heap

Post heap has been reordered

```
1 if (newNode not the root)
  1 set parent to parent of newNode
  2 if (newNode key > parent key)
    1 exchange newNode and parent)
    2 reheapUp (heap, parent)
  3 end if
2 end if
end reheapUp
```

ALGORITHM 9-2 Reheap Down

Algorithm reheapDown (heap, root, last)

Reestablishes heap by moving data in root down to its correct location in the heap.

Pre heap is an array of data

 root is root of heap or subheap

 last is an index to the last element in heap

Post heap has been restored

Determine which child has larger key

1 if (there is a left subtree)

 1 set leftKey to left subtree key

 2 if (there is a right subtree)

 1 set rightKey to right subtree key

 3 else

 1 set rightKey to null key

ALGORITHM 9-2 Reheap Down (continued)

```
4  end if
5  if (leftKey > rightKey)
    1  set largeSubtree to left subtree
6  else
    1  set largeSubtree to right subtree
7  end if
  Test if root > larger subtree
8  if (root key < largeSubtree key)
    1  exchange root and largeSubtree
    2  reheapDown (heap, largeSubtree, last)
9  end if
2  end if
end reheapDown
```


ALGORITHM 9-3 Build Heap

Algorithm buildHeap (heap, size)

Given an array, rearrange data so that they form a heap.

Pre heap is array containing data in nonheap order
 size is number of elements in array

Post array is now a heap

```
1 set walker to 1
2 loop (walker < size)
  1 reheapUp(heap, walker)
  2 increment walker
3 end loop
end buildHeap
```

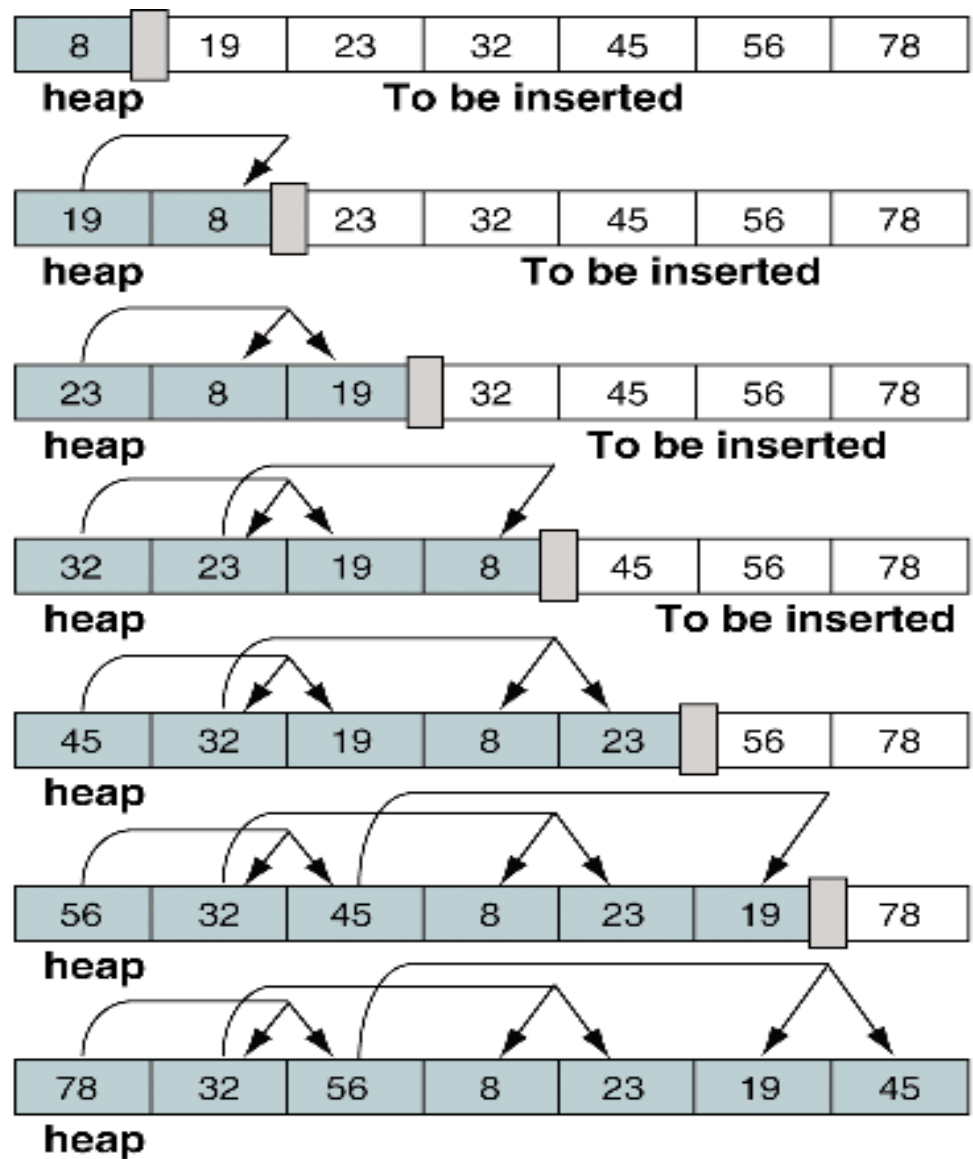
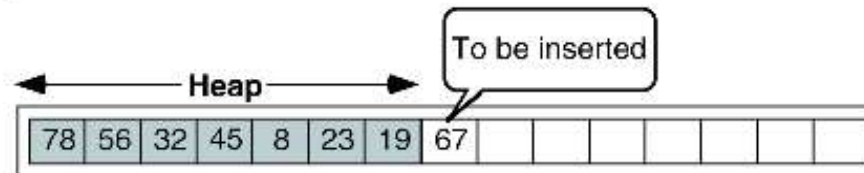
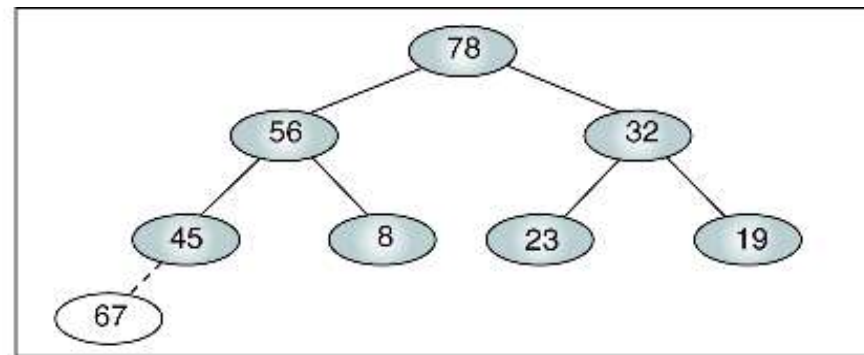
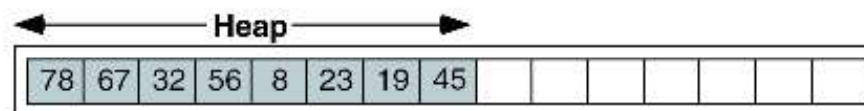
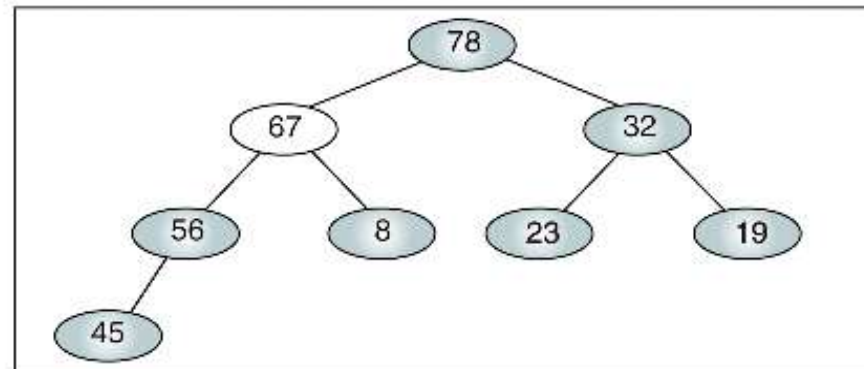


FIGURE 9-9 Building a Heap



(a) Before reheap up



(b) After reheap up

FIGURE 9-10 Insert Node

ALGORITHM 9-4 Insert Heap

Algorithm insertHeap (heap, last, data)

Inserts data into heap.

Pre heap is a valid heap structure

 last is reference parameter to last node in heap

 data contains data to be inserted

Post data have been inserted into heap

Return true if successful; false if array full

1 if (heap full)

 1 return false

2 end if

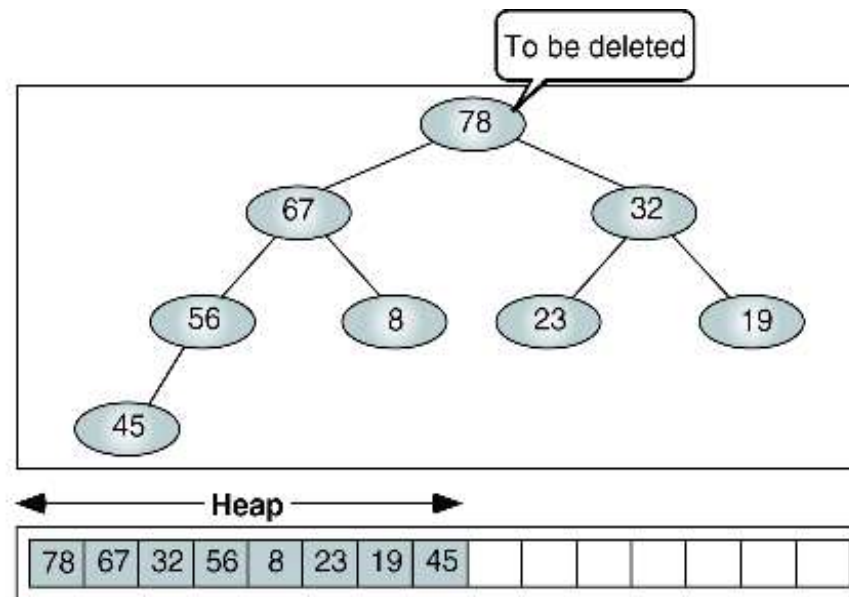
3 increment last

4 move data to last node

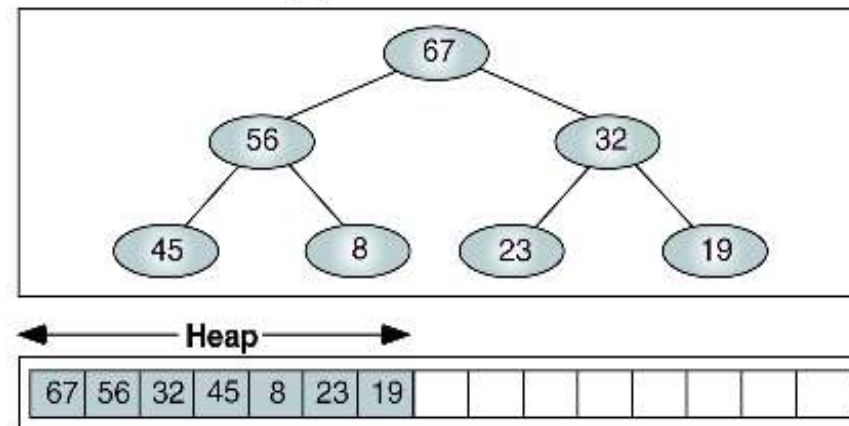
5 reheapUp (heap, last)

6 return true

end insertHeap



(a) Before delete



(b) After delete

FIGURE 9-11 deleteHeap Node

ALGORITHM 9-5 Delete Heap Node

Algorithm deleteHeap (heap, last, dataOut)

Deletes root of heap and passes data back to caller.

Pre heap is a valid heap structure

 last is reference parameter to last node in heap

 dataOut is reference parameter for output area

Post root deleted and heap rebuilt

 root data placed in dataOut

Return true if successful; false if array empty

1 if (heap empty)

 1 return false

2 end if

3 set dataOut to root data

4 move last data to root

5 decrement last

6 reheapDown (heap, 0, last)

7 return true

end deleteHeap

9-3 Heap ADT

We begin with a discussion of the heap ADT design and then develop the C code for the five major functions developed in Section 9.2.

- **Heap Structure**
- **Heap Algorithms**

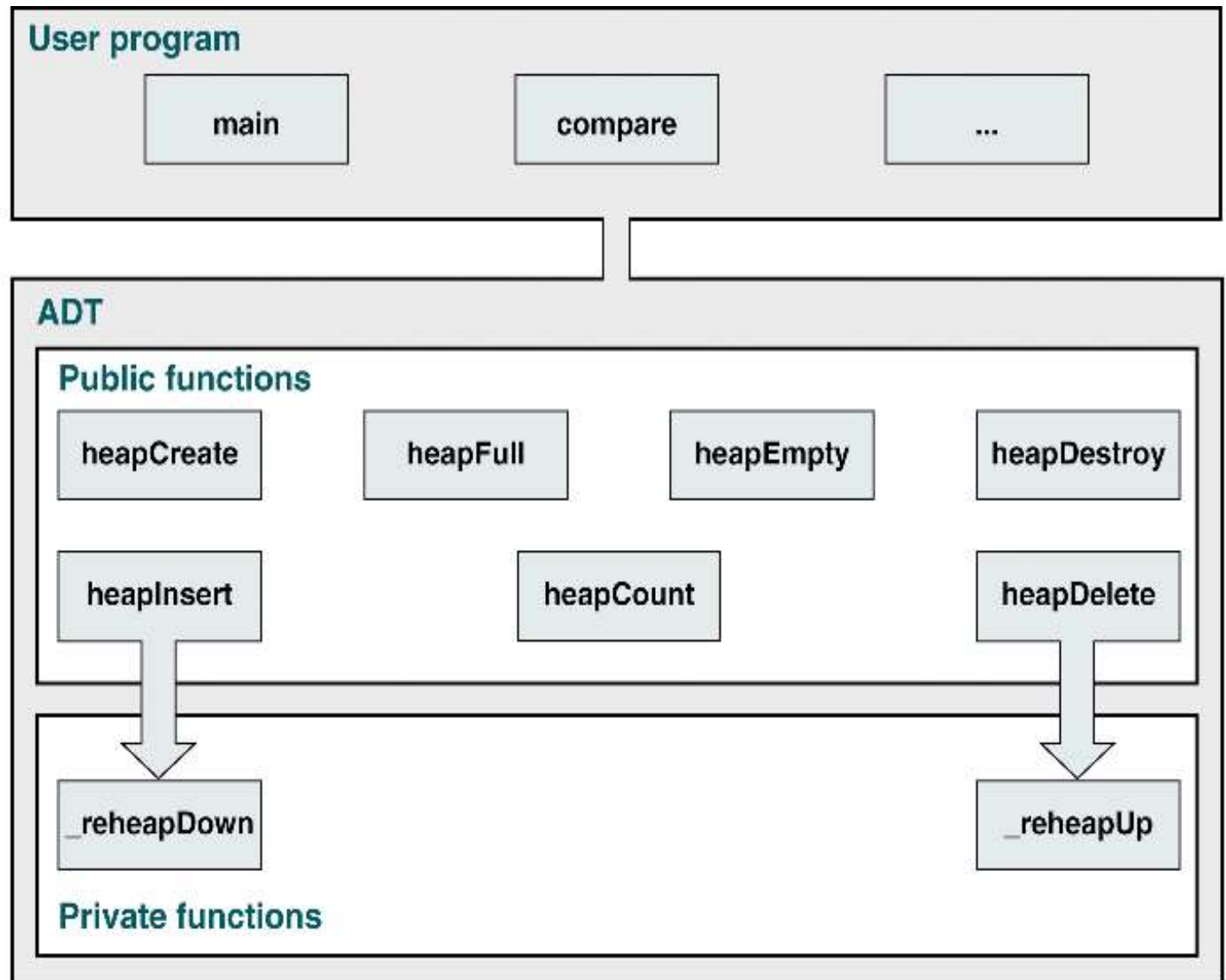


FIGURE 9-12 Heap ADT Design

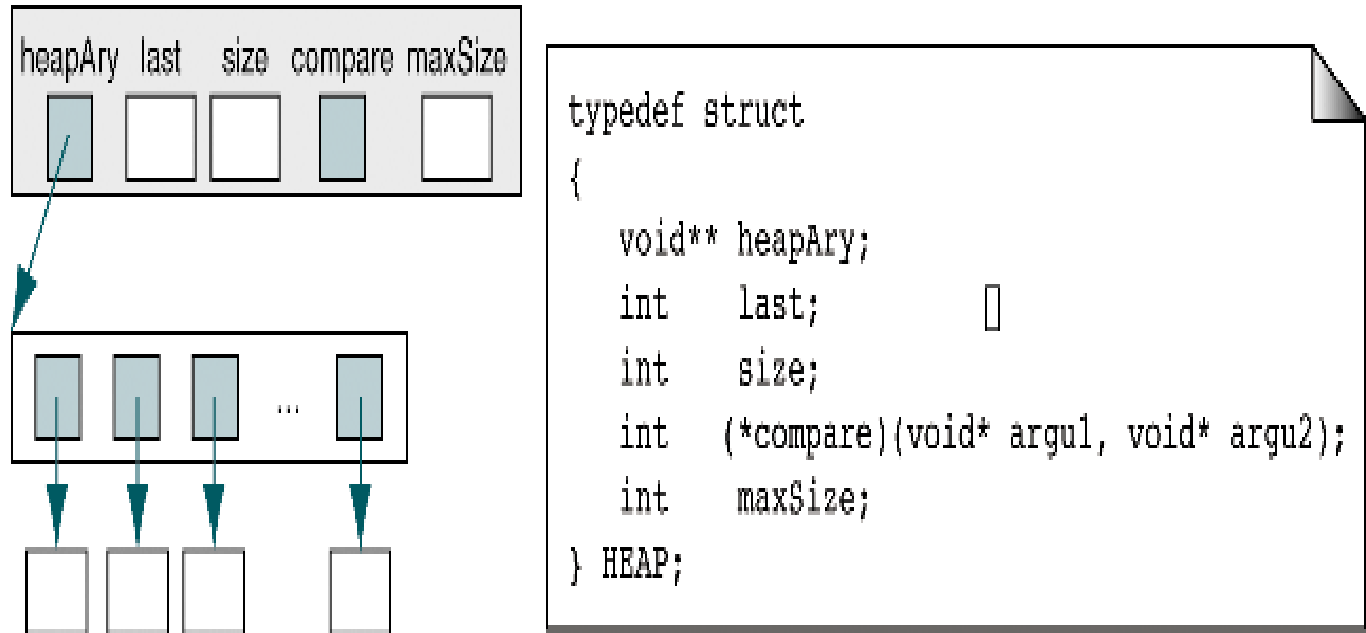


FIGURE 9-13 Heap ADT Structure

PROGRAM 9-1 Heap Declaration

```
1  /* Data Structures for heap ADT
2      Created by:
3      Date:
4  */
```

PROGRAM 9-1 Heap Declaration (*continued*)

```
5  #include <stdbool.h>
6
7  typedef struct
8  {
9      void** heapAry;
10     int    last;
11     int    size;
12     int    (*compare) (void* arg1, void* arg2);
13     int    maxSize;
14 } HEAP;
15
16 // Prototype Definitions
17 HEAP* heapCreate (int maxSize,
18                  int (*compare) (void* arg1, void* arg2));
19 bool  heapInsert  (HEAP* heap, void*  dataPtr);
20 bool  heapDelete  (HEAP* heap, void** dataOutPtr);
21 int   heapCount   (HEAP* heap);
22 bool  heapFull    (HEAP* heap);
23 bool  heapEmpty   (HEAP* heap);
24 void  heapDestroy (HEAP* heap);
25
26 static void _reheapUp   (HEAP* heap, int childLoc);
27 static void _reheapDown (HEAP* heap, int root);
```


PROGRAM 9-2 Create Heap Application Interface

```
1  /* ===== heapCreate =====
2  Allocates memory for heap and returns address of
3  heap head structure.
4      Pre  Nothing
5      Post heap created and address returned
6           if memory overflow, NULL returned
7  */
8  #include <math.h>
9
10 HEAP* heapCreate (int maxSize,
11                  int (*compare) (void* arg1, void* arg2))
12 {
13     // Local Definitions
14     HEAP* heap;
15
```

continued

PROGRAM 9-2 Create Heap Application Interface *(continued)*

```
16 // Statements
17 heap = (HEAP*)malloc(sizeof (HEAP));
18 if (!heap)
19     return NULL;
20
21 heap->last    = -1;
22 heap->compare = compare;
23
24 // Force heap size to power of 2 -1
25 heap->maxSize =
26     (int) pow (2, ceil(log2(maxSize))) - 1;
27 heap->heapAry = (void*)
28     calloc(heap->maxSize, sizeof(void*));
29 return heap;
30 } // createHeap
```

PROGRAM 9-3 Insert Heap Application Interface

```
1  /* ===== heapInsert =====
2     Inserts data into heap.
3     Pre      Heap is a valid heap structure
4              last is pointer to index for last element
5              data is data to be inserted
6     Post     data have been inserted into heap
7     Return true if successful; false if array full
8  */
9  bool heapInsert (HEAP* heap, void* dataPtr)
10 {
11     // Statements
12     if (heap->size == 0)                // Heap empty
13     {
14         heap->size                       = 1;
15         heap->last                       = 0;
16         heap->heapAry[heap->last] = dataPtr;
17         return true;
18     } // if
19     if (heap->last == heap->maxSize - 1)
20         return false;
21     ++(heap->last);
22     ++(heap->size);
23     heap->heapAry[heap->last] = dataPtr;
24     _reheapUp (heap, heap->last);
```

PROGRAM 9-3 Insert Heap Application Interface (continued)

continued

```
25     return true;
26 } // heapInsert
```

PROGRAM 9-4 Internal Reheap Up Function

```
1  /* ===== reheapUp =====
2     Reestablishes heap by moving data in child up to
3     correct location heap array.
4         Pre  heap is array containing an invalid heap
5             newNode is index to new data in heap
6         Post newNode inserted into heap
7  */
8  void _reheapUp (HEAP* heap, int childLoc)
9  {
10 // Local Definitions
11     int    parent;
12     void** heapAry;
13     void*  hold;
14
```

PROGRAM 9-4 Internal Reheap Up Function (continued)

```
15 // Statements
16 // if not at root of heap -- index 0
17 if (childLoc)
18 {
19     heapAry = heap->heapAry;
20     parent = (childLoc - 1) / 2;
21     if (heap->compare(heapAry[childLoc],
22                     heapAry[parent]) > 0)
23         // child is greater than parent -- swap
24         {
25             hold = heapAry[parent];
26             heapAry[parent] = heapAry[childLoc];
27             heapAry[childLoc] = hold;
28             _reheapUp (heap, parent);
29         } // if heap[]
30     } // if newNode
31     return;
32 } // reheapUp
```

PROGRAM 9-5 Delete Heap Application Interface

```
1  /* ===== heapDelete =====
2     Deletes root of heap and passes data back to caller.
3     Pre    heap is a valid heap structure
4            last is reference to last node in heap
5            dataOut is reference to output area
6     Post   last deleted and heap rebuilt
7            deleted data passed back to user
8     Return true if successful; false if array empty
9  */
10 bool heapDelete (HEAP* heap, void** dataOutPtr)
11 {
12     // Statements
13     if (heap->size == 0)
14         // heap empty
15         return false;
16     *dataOutPtr = heap->heapAry[0];
17     heap->heapAry[0] = heap->heapAry[heap->last];
18     (heap->last)--;
19     (heap->size)--;
20     _reheapDown (heap, 0);
21     return true;
22 } // heapDelete
```

PROGRAM 9-6 Internal Reheap Down Function

```
1  /* ===== reheapDown =====
2     Reestablishes heap by moving data in root down to its
3     correct location in the heap.
4     Pre  heap is array of data
5         root is root of heap or subheap
6         last is an index to last element in heap
7     Post heap has been restored
8  */
9  void _reheapDown (HEAP* heap, int root)
10 {
11 // Local Definitions
```

PROGRAM 9-6 Internal Reheap Down Function *(continued)*

```
12     void* hold;
13     void* leftData;
14     void* rightData;
15     int   largeLoc;
16     int   last;
17
18     // Statements
19     last = heap->last;
20     if ((root * 2 + 1) <= last)           // left subtree
21         // There is at least one child
22     {
```


PROGRAM 9-6 Internal Reheap Down Function (continued)

```
23     leftData    = heap->heapAry[root * 2 + 1];
24     if ((root * 2 + 2) <= last)    // right subtree
25         rightData = heap->heapAry[root * 2 + 2];
26     else
27         rightData = NULL;
28
29     // Determine which child is larger
30     if ((!rightData)
31         || heap->compare (leftData, rightData) > 0)
32     {
33         largeLoc = root * 2 + 1;
34     } // if no right key or leftKey greater
35     else
36     {
37         largeLoc = root * 2 + 2;
38     } // else
39     // Test if root > larger subtree
40     if (heap->compare (heap->heapAry[root],
41         heap->heapAry[largeLoc]) < 0)
42     {
43         // parent < children
44         hold = heap->heapAry[root];
45         heap->heapAry[root] =
46             heap->heapAry[largeLoc];
47         heap->heapAry[largeLoc] = hold;
48         _reheapDown (heap, largeLoc);
49     } // if root <
50 } // if root
51 return;
52 } // reheapDown
```